

JAVA SDE-2 INTERVIEW PREPARATION SERIES

SERIES INDEX - 18 STEPS / 28 BOOKS

Java SDE-2 Interview Preparation Series Index

A Basics-to-Advanced Navigation Guide

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

One ordered path through mathematical foundations, DSA patterns, Java engineering, production judgment, and SDE-2 interview practice.

FOUNDATIONS | DSA | JAVA | JVM | BACKEND | INTERVIEW LOOPS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

How to Use the Series

The website, PDF covers, roadmap pages, bookmarks, and previous/next links form one learning system. When rebuilding fundamentals, begin with Java Foundations (Study Step 01), continue to Time and Space Complexity (02), and then complete Number Systems foundations (03A) and its interview workbook (03B). For targeted revision, begin with the first study step whose readiness checks are not yet automatic.

Keep the PDFs together in the same directory so relative links can work in viewers that permit local-file navigation. The printed filenames remain the fallback when a viewer blocks those links.

Choose your starting point in 60 seconds

Start rule: choose the earliest row that describes a real gap. Do not begin with the most advanced-sounding topic; begin where your explanation or implementation first becomes unreliable.

Your current situation	Start here	Continue with
Never written Java	01 from Chapter 1	02, then 03A and 03B, then 04-17
Rebuilding from fundamentals	01	02, then 03A and 03B, then 04-17
Comfortable with Java syntax but weak in DSA	02	03A and 03B, then the first weak step in 04-17
Strong in DSA but weak in Java internals	18A	18B through 18F as needed
Preparing for a Java backend role	18B and 18D	18F, then 18H through 18J
Entering final interview revision	18G	Revisit only the gaps exposed by mocks

The learning loop for every volume

- 1 **Recognize:** use the entry signals and decision maps to identify the pattern.
- 2 **Understand:** restate the contract, invariant, and failure modes in your own words.
- 3 **Implement:** type the Java example and test boundaries before looking back.
- 4 **Explain:** give the complexity, trade-offs, and one production follow-up aloud.
- 5 **Prove readiness:** pass the completion check before opening the next volume.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

How to Use the Series	2
Part I - Foundation and DSA Learning Steps	5
Study Step 01 - Java Foundations for Problem Solving	5
Study Step 02 - Time and Space Complexity for Java Interviews	6
Study Step 03A - Number Systems and Math Foundations for DSA Interviews	7
Study Step 03B - Number Systems Interview Patterns and Rapid Revision	8
Study Step 04 - Bit Manipulation in Java	9
Study Step 05 - Loop Mastery, Patterns, and Index Calculations	10
Study Step 06 - Arrays and Array Problem-Solving Patterns	11
Study Step 07 - Strings and String Problem-Solving Patterns	12
Study Step 08 - Hashing: Maps, Sets, Frequency, and Prefix State	13
Study Step 09 - Recursion and Backtracking in Java	14
Study Step 10 - Linked Lists: Pointer Reasoning and Mutation	15
Study Step 11 - Stacks, Queues, Deques, and Monotonic Patterns	16
Study Step 12 - Binary Search: Bounds, Answers, and Invariants	17
Study Step 13 - Trees, Binary Search Trees, and Tries	18
Study Step 14 - Heaps, Priority Queues, Selection, and Top-K	19
Study Step 15 - Graphs: Traversal, Ordering, Paths, and Union-Find	20
Study Step 16 - Greedy Algorithms: Recognition, Proofs, and Scheduling	21
Study Step 17 - Dynamic Programming: State, Transitions, and Optimization	22

CONTENTS NAVIGATION

Part II - Advanced Java Collection	23
Study Step 18A - Advanced Java A: JVM and Execution	23
Study Step 18B - Advanced Java B: Language, OOP, and Modern Java	24
Study Step 18C - Advanced Java C: Collections, Streams, and I/O	25
Study Step 18D - Advanced Java D: Concurrency and the Memory Model	26
Study Step 18E - Advanced Java E: Performance, Diagnostics, and GC Incidents	27
Study Step 18F - Advanced Java F: Design, Backend, Testing, and Security	28
Study Step 18G - Advanced Java G: Question Bank, Study Plan, and Reference	29
Study Step 18H - Advanced Java H: Spring Boot and REST APIs	30
Study Step 18I - Advanced Java I: Persistence, SQL, JPA, and Caching	31
Study Step 18J - Advanced Java J: Distributed Systems and System Design	32
Part III - Recommended Study Rhythm	33
Build, Practice, Explain, Revisit	33

Part I - Foundation and DSA Learning Steps

Study Step 01 - Java Foundations for Problem Solving

This volume is the canonical Java language foundation and the recommended first book in the series. It starts with source-to-execution and program structure, builds through types, control flow, methods, arrays, strings, objects, core libraries, exceptions, generics, and usage-first collections, then converts that knowledge into interview-quality implementation and debugging skill.

Open [Java-SDE2-DSA-03-Java-Foundations-for-Problem-Solving.pdf](#).

Study Step 02 - Time and Space Complexity for Java Interviews

This volume is the bridge from Java syntax to algorithmic problem solving. It begins with concrete operation counts, introduces growth and space gently, then develops the qualified cost models, constraints, invariants, amortization, and optimization explanations expected in SDE-2 interviews.

Open [Java-SDE2-DSA-02-Time-and-Space-Complexity.pdf](#).

Study Step 03A - Number Systems and Math Foundations for DSA Interviews

This volume closes the mathematical prerequisite gap before bit manipulation, arrays, binary search, hashing, and number-based coding problems. It is practical DSA mathematics, not an academic number-theory textbook.

Open [Java-SDE2-DSA-01-Number-Systems-and-Math-Foundations.pdf](#).

Study Step 03B - Number Systems Interview Patterns and Rapid Revision

This linked workbook completes Study Step 03 with the 52-implementation catalog, Java number traps, conceptual and code-output questions, debugging tasks, coding exercises, solutions, follow-up chains, cheat sheet, and readiness assessment.

Open [Java-SDE2-DSA-01B-Number-Systems-Interview-Workbook.pdf](#).

Study Step 04 - Bit Manipulation in Java

This dedicated volume turns the binary foundation from Study Step 03A into a progressive Java interview toolkit. It begins with visible bit patterns and safe operators, derives each shortcut instead of asking readers to memorize it, then develops XOR families, subset state, SDE-2 problem patterns, Java APIs, and production-quality bit fields.

Open [Java-SDE2-DSA-04-Bit-Manipulation-in-Java.pdf](#).

Study Step 05 - Loop Mastery, Patterns, and Index Calculations

This volume is the bridge from basic Java control flow to reliable array and string problem solving. It begins with the execution model of for, while, do-while, and enhanced-for loops; builds a visual language for ranges and invariants; then derives pointer, compaction, window, search, and matrix patterns without asking the reader to memorize templates blindly.

Open [Java-SDE2-DSA-05-Loop-Mastery-and-Index-Calculations.pdf](#).

Study Step 06 - Arrays and Array Problem-Solving Patterns

Arrays are the default interview state space. This volume begins with storage, indexes, references, copying, and traversal before building a constraint-driven pattern map for reliable SDE-2 solutions.

Open [Java-SDE2-DSA-06-Arrays-and-Array-Patterns.pdf](#).

Study Step 07 - Strings and String Problem-Solving Patterns

This prerequisite-first volume takes a reader from SDE-1 String mechanics through the substring, frequency, window, parsing, and exact-search patterns expected in SDE-2 interviews. Every advanced technique follows a baseline, invariant, dry run, Java implementation, failure boundary, and complete complexity discussion.

Open [Java-SDE2-DSA-07-Strings-and-String-Patterns.pdf](#).

Study Step 08 - Hashing: Maps, Sets, Frequency, and Prefix State

Hashing is both a DSA pattern and a Java object-contract problem. This volume connects the two so expected $O(1)$ claims remain defensible.

Open [Java-SDE2-DSA-08-Hashing-Maps-Sets-and-Prefix-State.pdf](#).

Study Step 09 - Recursion and Backtracking in Java

This volume separates recursion as a method contract from backtracking as controlled state-space search, with Java stack and mutation behavior made explicit.

Open [Java-SDE2-DSA-09-Recursion-and-Backtracking.pdf](#).

Study Step 10 - Linked Lists: Pointer Reasoning and Mutation

Linked-list problems expose whether a candidate can preserve reachability while mutating references. This volume makes those invariants visible.

Open [Java-SDE2-DSA-10-Linked-Lists.pdf](#).

Study Step 11 - Stacks, Queues, Deques, and Monotonic Patterns

This volume unifies ordering data structures around the state each one preserves, then applies monotonic structures to linear-time interview patterns.

Open [Java-SDE2-DSA-11-Stacks-Queues-Deques-and-Monotonic-Patterns.pdf](#).

Study Step 12 - Binary Search: Bounds, Answers, and Invariants

Binary search is a proof pattern over monotone state. This volume centers interval semantics, duplicates, insertion points, overflow-safe midpoint calculation, and search on answers.

Open [Java-SDE2-DSA-12-Binary-Search.pdf](#).

Study Step 13 - Trees, Binary Search Trees, and Tries

Tree interviews test recursion, state ownership, structural invariants, and the ability to translate ordering into pruning.

Open [Java-SDE2-DSA-13-Trees-BSTs-and-Tries.pdf](#).

Study Step 14 - Heaps, Priority Queues, Selection, and Top-K

This volume distinguishes heap order from global sorting and develops priority-based interview patterns without hiding comparator or mutation risks.

Open [Java-SDE2-DSA-14-Heaps-Priority-Queues-and-Top-K.pdf](#).

Study Step 15 - Graphs: Traversal, Ordering, Paths, and Union-Find

Graph problems become manageable once the model and edge semantics are explicit. This volume emphasizes algorithm selection, state, overflow, and correctness.

Open [Java-SDE2-DSA-15-Graphs.pdf](#).

Study Step 16 - Greedy Algorithms: Recognition, Proofs, and Scheduling

Greedy solutions are short to code and hard to justify. This volume centers the proof obligation, not the superficial loop.

Open [Java-SDE2-DSA-16-Greedy-Algorithms.pdf](#).

Study Step 17 - Dynamic Programming: State, Transitions, and Optimization

This volume makes dynamic programming a specification process: define exactly what a state means, how it depends on smaller states, and what information can be compressed.

Open [Java-SDE2-DSA-17-Dynamic-Programming.pdf](#).

Part II - Advanced Java Collection

Study Step 18A - Advanced Java A: JVM and Execution

Part A builds the execution model needed to reason about Java behavior beneath syntax without presenting implementation details as language guarantees.

Open [Java-SDE2-ADV-18A-JVM-and-Execution.pdf](#).

Study Step 18B - Advanced Java B: Language, OOP, and Modern Java

Part B covers the language-engineering depth expected when SDE-2 interviews move beyond coding patterns into API design and type safety.

Open [Java-SDE2-ADV-18B-Language-OOP-and-Modern-Java.pdf](#).

Study Step 18C - Advanced Java C: Collections, Streams, and I/O

Part C is the advanced Java-library companion to the DSA volumes, emphasizing API contracts, representation costs, resource ownership, and production boundaries.

Open [Java-SDE2-ADV-18C-Collections-Streams-and-IO.pdf](#).

Study Step 18D - Advanced Java D: Concurrency and the Memory Model

Part D develops concurrency correctness from first principles and then connects it to production scheduling, cancellation, backpressure, and testing.

Open [Java-SDE2-ADV-18D-Concurrency-and-Memory-Model.pdf](#).

Study Step 18E - Advanced Java E: Performance, Diagnostics, and GC Incidents

Part E teaches an evidence-first diagnostic method so performance claims and tuning actions remain falsifiable and operationally safe.

Open [Java-SDE2-ADV-18E-Performance-Diagnostics-and-GC.pdf](#).

Study Step 18F - Advanced Java F: Design, Backend, Testing, and Security

Part F covers the engineering judgment that distinguishes SDE-2 candidates from syntax-only candidates: boundaries, invariants, failure policy, observability, and maintainability.

Open [Java-SDE2-ADV-18F-Design-Backend-Testing-and-Security.pdf](#).

Study Step 18G - Advanced Java G: Question Bank, Study Plan, and Reference

Part G is the core Java revision system. It consolidates interview questions, selected solutions, study loops, terminology, and primary references without replacing deliberate practice.

Open [Java-SDE2-ADV-18G-Question-Bank-Study-Plan-and-Reference.pdf](#).

Study Step 18H - Advanced Java H: Spring Boot and REST APIs

Part H opens the backend specialist track with the Spring and HTTP reasoning expected in Java SDE-2 interviews, while keeping framework behavior tied to explicit contracts and failure modes.

Open [Java-SDE2-ADV-18H-Spring-Boot-and-REST.pdf](#).

Study Step 18I - Advanced Java I: Persistence, SQL, JPA, and Caching

Part I develops the persistence judgment needed to design correct, measurable Java data paths instead of treating databases, ORMs, and caches as interchangeable storage APIs.

Open [Java-SDE2-ADV-18I-Persistence-SQL-and-Caching.pdf](#).

Study Step 18J - Advanced Java J: Distributed Systems and System Design

Part J completes the backend specialist track with a constraint-first system-design method spanning distributed data, messaging, failure control, and operations.

Open [Java-SDE2-ADV-18J-Distributed-Systems-and-System-Design.pdf](#).

Part III - Recommended Study Rhythm

Build, Practice, Explain, Revisit

For each volume, complete one learning pass, one no-notes implementation pass, one spoken explanation pass, and one delayed revision pass. Keep an error log organized by contract, invariant, numeric safety, data-structure choice, complexity, and test coverage. Use the advanced collection only after the algorithmic foundation is stable or when a role requires deeper Java engineering immediately.

Series Roadmap

Complete the study steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. The same public study codes appear on the website and every PDF cover. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
01	Java Foundations for Problem Solving
02	Time and Space Complexity for Java Interviews
03A	Number Systems and Math Foundations for DSA Interviews
03B	Number Systems Interview Patterns and Rapid Revision
04	Bit Manipulation in Java
05	Loop Mastery, Patterns, and Index Calculations
06	Arrays and Array Problem-Solving Patterns
07	Strings and String Problem-Solving Patterns
08	Hashing: Maps, Sets, Frequency, and Prefix State
09	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18A	Advanced Java A: JVM and Execution
18B	Advanced Java B: Language, OOP, and Modern Java
18C	Advanced Java C: Collections, Streams, and I/O
18D	Advanced Java D: Concurrency and the Memory Model
18E	Advanced Java E: Performance, Diagnostics, and GC Incidents
18F	Advanced Java F: Design, Backend, Testing, and Security
18G	Advanced Java G: Question Bank, Study Plan, and Reference
18H	Advanced Java H: Spring Boot and REST APIs
18I	Advanced Java I: Persistence, SQL, JPA, and Caching
18J	Advanced Java J: Distributed Systems and System Design

Study Step 03 uses linked foundations (03A) and workbook (03B) books. Study Step 18 uses ten focused books (18A-18J), including a three-book backend specialist track. These suffixes keep every file focused while preserving one unambiguous route.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Series Index - 18 Steps / 28 Books. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.